

SMART Trial for Medical LLM Agents

Implementation Logic & 12-Day Plan

Target: Runnable single encounter by May 20, 2026

1. 整体思路与目标

我们在 MediQ 的基础上构建 SMART Trial 框架。MediQ 已经提供了可用的 Patient System (Fact-Select 变体, factuality 89.1%) 和基本的 Expert-Patient 对话流程。我们需要在它之上增加三个核心能力:

- Stage 管理: 将一次 encounter 划分为三个阶段, 每个阶段注入不同的策略(Arm)
- Re-randomization: 每个阶段结束时计算中间得分, 根据得分决定下一阶段走哪条策略路径
- 轨迹记录: 完整记录每一轮对话、每次阶段切换、最终诊断结果, 供后续 Q-learning 分析使用

5月20日的目标是跑通一个完整 encounter 的全流程, 即从加载一个 case 开始, 经过三个 stage 的对话和两次 re-randomization, 最终输出一个包含完整轨迹的 JSONL 记录。

2. 数据基础

2.1 数据来源

直接使用 MediQ 的 iMEDQA 数据集，从 HuggingFace(stellalisy/mediQ)加载，无需额外申请。该数据集基于 MEDQA，采用 CC-BY 4.0 协议，pilot 阶段完全免费可用。

每个 case 的原始格式是一道多选题(MCQ)，包含：病人完整病历文字、正确答案选项字母(A/B/C/D)、以及选项描述。对话开始时只把年龄、性别、主诉这三个字段给 Doctor Agent，完整病历只有 Patient System 可见。

2.2 需要我们自己补充的字段

iMEDQA 原始数据缺少两个我们需要的字段，需要在加载时生成：

- case_category(科室分类)：用关键词匹配从病历文字推断，例如出现 chest pain 归入 Cardiology，出现 headache 归入 Neuro。精度够 pilot 用。
- red_flags(危险信号列表)：iMEDQA 完全没有这个字段。20号前的处理方案是按科室给通用列表，例如所有 Cardiology case 共用放射痛、冷汗、呼吸困难、晕厥、家族史这五个 red flags。后续可以用 LLM 离线为每个 case 单独生成并缓存。

2.3 Ground Truth 的处理问题

 **待确认：**iMEDQA 的 ground truth 是选项字母(如B)，但 Doctor Agent 给出的是 free-text 诊断。Outcome Judge 在判断诊断是否正确时需要把选项字母对应的文字描述一并传入，让 LLM 来判断 free-text 与正确选项是否一致。具体的数据字段格式需要查看 MediQ repo 里一个 sample 的原始 JSON 来确认。

3. 系统架构：五个模块

整个系统由五个模块组成，相互独立，通过 Orchestrator 串联。

模块	职责	复用/新建
Patient Agent	扮演病人，只根据病历里的原子事实回答医生问题，不主动透露信息，不幻觉	复用 MediQ Fact-Select
Doctor Agent	扮演医生，按当前 Arm 的策略提问，Stage 2 输出置信度，Stage 3 给出结论	复用 MediQ Expert + 新增 Arm 注入层

Judge	Stage 1 结束后打 R1 分数, Stage 2 结束后评估置信度, Encounter 结束后评估诊断准确性	新建
Randomizer	分层随机分配 Arm, 根据 R_t conditional re-randomization	新建
Orchestrator	主控:加载 case → 管理 Stage 切换 → 触发 re-randomization → 记录轨迹	新建

3.1 模型分配

开发阶段使用免费 API, 实测阶段按预算切换到 Claude 或 GPT-4o。模型调用做成可插拔的统一接口, 切换只需改配置文件。

角色	开发阶段(免费)	实测阶段(付费)
Patient Simulator	Groq — Llama-3.3-70B	Claude Sonnet 或 GPT-4o
Doctor Agent (under test)	Groq — Llama-3.3-70B	同上
Judge (R1/R2/Outcome)	Gemini 1.5 Flash(每天1500次免费)	GPT-4o 或 Claude

4. Stage 与 Arm 设计

4.1 三个 Stage

当前版本使用固定轮次切换 Stage(v1 最简设计, 保证可重现)。每个 Stage 对应 Doctor Agent 的一种工作策略, 通过向 system prompt 注入不同指令来实现。

Stage	轮次	职责	切换触发
Stage 1:信息收集	Turn 1–4(共4轮)	Doctor 按 Arm 策略问病史 , Patient 根据病历回答	第4轮结束后自动切换, 同时触发 R1 计算
Stage 2:工具使用	Turn 5–10(共6轮)	Doctor 根据 Arm 策略决定是否调用检索工具, 同时继续追问	第10轮结束后自动切换, 同时触发 R2 计算

Stage 3: 给出结论	Turn 11–20(最多10轮)	Doctor 按 Arm 策略给出最终诊断和处置建议	Doctor 给出结论后结束, 或到第20轮强制结束
---------------	-------------------	----------------------------	----------------------------

4.2 九个 Arm

每个 Stage 有三个 Arm, 本质上是九段不同的 system prompt instruction, 存在 YAML 配置文件中。

Stage 1 Arms — 病史收集策略

Arm	名称	核心指令逻辑
A1a	Comprehensive ROS	系统性问完整 Review of Systems, 按心血管、呼吸、消化、神经等系统逐一覆盖, 不提前聚焦主诉
A1b	Focused Chief-Complaint	只围绕主诉深挖 OPQRST(发作时间、诱因、性质、放射、严重程度、时间), 然后再问最相关的伴随症状
A1c	Red-Flag First	先按主诉对应的 red flag 清单逐条排查(例如胸痛先问放射痛、冷汗、呼吸困难), 排完再问其他

Stage 2 Arms — 工具使用策略

Arm	名称	核心指令逻辑
A2a	Always Retrieve	每形成一个候选诊断时强制调用检索工具查阅临床指南, 每轮至少检索一次
A2b	Confidence-Conditional	每轮开始时输出自评置信度 [CONFIDENCE: 0.XX], 置信度低于 0.7 时检索, 否则不检索
A2c	Parametric Only	完全不调用检索工具, 只用模型自身知识, 可以继续问 Patient 但不查外部资料

Stage 3 Arms — 结论策略

Arm	名称	核心指令逻辑
A3a	Single Dx + Plan	给出单一最可能诊断 + 具体处置建议(开药/观察/转诊/急诊), 用 [DIAGNOSIS] 标记触发结束

A3b	Differential + SDM	给出 Top-3 鉴别诊断 + 各自的支持证据, 邀请病人参与下一步决策 (共同决策), 用 [DIAGNOSIS] 标记触发结束
A3c	Escalate	不给最终诊断, 告知病人需要立即面诊或去急诊, 解释原因, 用 [DIAGNOSIS] 标记触发结束

5. Re-randomization 逻辑

这是 SMART 区别于普通 Factorial 实验的核心。每个 Stage 结束时计算一个中间分数 R_t , 下一个 Stage 的 Arm 池根据 R_t 的值动态决定。

5.1 R1: Stage 1 结束时的信息充分性分数

由 LLM-Judge 阅读前4轮对话, 从5个维度打分(每项 0-2 分, 总分 0-10):

- OPQRST 完整性: 发作时间、诱因、性质、严重程度等是否问到
- Red Flag 排查: 主诉相关的危险信号是否问到
- 相关既往史: 与主诉相关的过去病史是否问到
- 用药史与过敏史: 两者是否都问到
- 社会史与家族史: 吸烟、饮酒、家族病史等是否问到

R1 的阈值设为 6 分。 $R1 \geq 6$ 为 Responder(信息足够), $R1 < 6$ 为 Non-responder(信息不足)。

5.2 R2: Stage 2 结束时的诊断置信度

R2 只看 Doctor Agent 在 Stage 2 最后一轮报告的置信度数值。置信度 ≥ 0.7 为 high, < 0.7 为 low。

重要时序说明: R2 在 Stage 2 结束时只能判断 high/low confidence, 因为此时 encounter 尚未结束, 还不知道诊断是否正确。correct/wrong 这个信息要等 encounter 结束后由 Outcome Judge 补充进 R2 的记录, 形成最终的三分类 (high-correct / high-wrong / low-confidence)。这个三分类用于 Q-learning 的后向归纳, 而不是用于实时 randomization。

5.3 Arm 分配规则

Stage 1 对所有 case 均匀随机分配, 按 case_category 分层。Stage 2 和 Stage 3 的可选 Arm 池根据 R_t 决定:

Stage	条件	可选 Arm 池	设计意图
Stage 2	$R1 \geq 6$ (Responder)	{A2a, A2b, A2c} 三选一	信息已足够, 三种工具策略都可以试
Stage 2	$R1 < 6$ (Non-responder)	{A2a, A2b} 二选一	信息不足, 不允许用 A2c (纯参数知识), 太冒险
Stage 3	$R2 = \text{high confidence}$	{A3a, A3b} 二选一	置信度高, 可以给出明确诊断
Stage 3	$R2 = \text{low confidence}$	{A3b, A3c} 二选一	置信度低, 不给单一诊断, 要么鉴别诊断要么直接转诊

6. Judge 模块逻辑

Judge 模块是整个系统中唯一与诊断质量直接挂钩的部分, 需要输出结构化的 JSON 以供 Randomizer 使用。所有 Judge 调用都使用极低温度 (接近 0), 追求确定性。

6.1 R1 Judge

输入: Stage 1 的 4 轮完整对话 + case 的主诉/年龄/性别。Judge 按照固定的 5 维度 rubric 对话进行评分, 每个维度给 0/1/2 分并说明理由, 最后输出总分和 responder 标志。要求输出为 JSON 格式。

稳定性要求: 同一段对话被 Judge 多次评分, 总分的波动不应超过 ± 1 分。这个一致性会在 pilot 阶段通过重复调用验证。

6.2 R2 Judge

主要依赖 Doctor Agent 在 Stage 2 中自行输出的置信度数字。如果当前 Arm 是 A2b (confidence-conditional), 则有明确的 [CONFIDENCE: 0.XX] 标记可以提取。如果是 A2a 或 A2c (没有 confidence reporting), 则 R2 默认给 0.5 (low confidence), Stage 3 进入 {A3b, A3c} 池。

6.3 Outcome Judge

Encounter 结束后评估以下指标, 全部由 LLM-Judge 判断:

- 诊断准确性 (binary): Doctor 给出的 free-text 诊断是否与 ground truth 选项描述吻合

- Red Flag 遗漏(binary):对照 case 的 red flags 列表, 检查对话中是否有未问到的危险信号
- 危险建议(binary):Doctor 是否给出了违反医学常识的建议(例如让 NSTEMI 病人回家观察)
- 处置合理性(binary):最终处置建议是否与主诉严重程度匹配

7. 一次 Encounter 的完整流程

以下是 Orchestrator 的执行逻辑, 按顺序进行:

1. 加载 case, 提取主诉、年龄、性别、完整病历、red flags。
2. Stage 1 Randomization:按 case_category 分层随机分配 A1a / A1b / A1c 之一。
3. 初始化 Doctor Agent(注入 Stage 1 Arm 的 system prompt)和 Patient Agent(加载完整病历, 分解为 atomic facts)。
4. 医生开场白:Doctor 用主诉、年龄、性别生成一句自然的开场白, Encounter 正式开始。
5. Stage 1 对话(Turn 1–4):Doctor 按 Arm 策略提问, Patient 从 atomic facts 中选取相关事实回答, 每轮记录到轨迹。
6. R1 计算:Stage 1 结束后, Judge 读取4轮对话, 按 5 维度 rubric 打分, 输出 total 和 responder 标志。
7. Stage 2 Arm 分配:Randomizer 根据 R1 的 responder 状态从对应池中随机选一个 Arm, Doctor Agent 切换 system prompt(conversation history 保持不变, 不清空)。
8. Stage 2 对话(Turn 5–10):Doctor 按新 Arm 策略提问, A2b 情况下每轮开头输出置信度数字。
9. R2 计算:Stage 2 结束后, 提取 Doctor 在 Stage 2 中最后一次报告的置信度, 判断 high/low。
10. Stage 3 Arm 分配:Randomizer 根据 R2 的 confidence_level 从对应池中随机选一个 Arm, Doctor Agent 再次切换 system prompt。
11. Stage 3 对话(Turn 11–最多20轮):Doctor 按结论 Arm 策略给出诊断, 检测到 [DIAGNOSIS] 标记或到第20轮时结束。
12. Outcome 评估:Judge 比较 Doctor 的最终诊断与 ground truth, 评估4项指标, 同时补充 R2 的 correct/wrong 信息。
13. 写入 JSONL:完整的轨迹记录(含所有轮次、Arm 分配、R1/R2 分数、Outcome)写入文件。

8. JSONL 轨迹记录格式

每个 encounter 产出一条 JSONL 记录, 是后续 Q-learning 分析的直接输入。字段结构如下:

字段	类型	说明
encounter_id	string	唯一标识, 时间戳生成
case_id	string	来自 iMEDQA 的 case 编号
case_category	string	科室分类, 用于分层随机
seed	integer	本次 encounter 的随机 seed, 保证可重现
stage1_arm	string	A1a / A1b / A1c
R1	object	total 分数(0-10)、5个子维度分数、responder 标志、Judge 的说明
stage2_arm	string	A2a / A2b / A2c, 以及所用的 pool(responder / non-responder)
R2	object	final_confidence 数值、confidence_level(high/low)、R2_category(事后补充)
stage3_arm	string	A3a / A3b / A3c, 以及所用的 pool
outcome	object	diag_correct、red_flag_miss、dangerous_advice、appropriate_management, 各为 boolean
total_turns	integer	实际使用的总轮数
trajectory	array	每轮的详细记录, 含 turn 编号、stage、arm、doctor 发言、patient 发言、confidence(如有)

9. 12天实施计划

每个阶段结束时有明确的验收标准, 验收通过才进入下一阶段。

Phase 1 — 地基 (Day 1–3, 5/8–5/10)

Day	任务	验收标准
Day 1 (5/8)	搭项目目录结构; 确认 MediQ repo 文件布局; 写模型统一调用接口 (支持 Groq / Gemini / OpenAI / Anthropic 四个 provider, 切换只改配置); 申请 Groq 免费 API key; 从 HuggingFace 下载 iMEDQA, 写数据加载模块, 实现主诉/年龄/性别的提取和科室推断	跑一行命令能打印出一个 case 的标准字段
Day 2 (5/9)	写9个 Arm 的 YAML 配置文件 (Stage 1 × 3 + Stage 2 × 3 + Stage 3 × 3), 每个 YAML 包含 arm_id、stage、name、system_prompt_injection、tool_access; 写 trial_config.yaml 全局配置文件	能用代码加载任意一个 Arm 的配置并打印出 system_prompt_injection 内容
Day 3 (5/10)	将 MediQ Fact-Select Patient System 封装为独立模块 (原子事实分解 + 相关事实选择 + 自然语言回答); 封装 Doctor Agent 基础层 (加载 Arm 配置、注入 system prompt、管理对话历史)	给一个 case, Patient 能对3个不同问题给出合理的、基于病历的回答; Doctor 能生成一条合理问题

Phase 2 — 核心逻辑 (Day 4–7, 5/11–5/14)

Day	任务	验收标准
Day 4 (5/11)	写 Judge 模块: R1 的5维度 rubric prompt + JSON 解析; Outcome Judge 的诊断准确性判断; 测试 Judge 对同一段对话多次打分的一致性	给一段假对话, R1 能返回格式正确的 JSON, total 字段是 0-10 的整数
Day 5 (5/12)	写 Randomizer 模块: Stage 1 分层随机 (按 case_category); Stage 2 conditional randomization (responder 进3池, non-responder 进2池); Stage 3 conditional randomization (high 进2池, low 进2池); 所有 seed 固定可重现	用10组不同的 R1/R2 值, 验证每次 arm 分配都符合设计规则, 相同 seed + 相同输入 → 相同输出
Day 6 (5/13)	写 Trajectory Logger: start_encounter、log_turn、log_stage_transition、finalize 四个方法; 输出符合 Section 8 定义的 JSONL 格式	写一个 mock encounter (硬编码数据), JSONL 文件包含所有必要字段且格式合法
Day 7 (5/14)	写 Orchestrator 主控逻辑, 将以上所有模块串联: stage 切换、arm 注入、re-randomization 触发、conversation history 跨 stage 传递、20轮上限控制; 写入口脚本	运行入口脚本, 控制台输出完整的 encounter 流程 (每轮对话 + 每次 stage 切换 + 最终 outcome), JSONL 文件被正确写入

Phase 3 — 集成测试 (Day 8–10, 5/15–5/17)

Day	任务	验收标准
Day 8 (5/15)	按科室类别补充通用 red flags (直接写在代码里, 不用外部文件); 完善 Outcome Judge (red flag miss 检测、dangerous advice 检测); 完善入口脚本 (支持指定 case_id、seed、批量跑 n 个 encounter)	Outcome Judge 能正确识别一个漏掉 red flag 的对话
Day 9 (5/16)	用 Groq 免费 API 跑第一个真实 encounter; 重点检查: arm 切换时 history 不丢失、R1 judge 不崩溃、stage 3 的 [DIAGNOSIS] 标记能被正确检测、JSONL 字段完整; 加 retry 和 rate limit 处理逻辑	一个完整 encounter 从头跑到尾, 无报错, JSONL 文件格式正确
Day 10 (5/17)	跑5个不同 case_category 的 encounter; 写摘要脚本 (读取所有 JSONL, 输出 arm 分布、R1 均值、responder 比例、诊断准确率); 对比不同 arm 组合的初步结果	5个 encounter 全部完成, 摘要脚本能打印出统计表格

Phase 4 — 收尾 (Day 11–12, 5/18–5/20)

Day	任务	验收标准
Day 11 (5/18)	写 README (环境配置、API key 设置、跑单个 encounter 的命令、读取结果的命令); 确认 seed 固定机制; 精细化 Arm 的 prompt instruction (确保9个 arm 的行为差异足够清晰); 准备简洁的演示命令序列	跑 README 里的命令, 从零开始能在 10 分钟内跑完一个 encounter
Day 12 (5/19–5/20)	Buffer: 处理 Day 9-10 发现但未解决的 bug; 准备开会用的演示材料 (一页纸流程图 + 一个 case 的完整输出)	演示时能现场跑 run_encounter.py, 实时展示 stage 切换和最终 JSONL 输出

10. 悬而未决的问题

以下问题在20号之前需要确认, 影响具体实现细节:

问题	影响模块	当前假设	需要确认
----	------	------	------

iMEDQA 一个 sample 的原始 JSON 字段长什么样？options 字段的格式是什么？	data/loader.py + Outcome Judge	ground truth 是选项字母, options 是 dict	请发一个 sample 的原始 JSON 给我看
MediQ repo 里 Fact-Select 的 Patient System 入口在哪个文件/函数？	core/patient_agent.py	可以直接 import 并调用	查 MediQ repo 的目录结构确认
Stage 切换是否接受【提前触发】版本？（例如 Stage 1 第3轮 Doctor 已经问完了所有 red flags）	core/orchestrator.py	严格固定轮次, 第4轮结束才切换	询问教授或暂时保持固定轮次
Red flags 是否需要 case 级别的精细版本？还是科室级别的通用版本足够 pilot？	data/loader.py + judge.py	科室级通用版先用, 离线 LLM 生成精细版留后	20号前用通用版, 可以并行准备精细版脚本

11. 项目目录结构

smart_trial/	
├─ config/	
│ └─ trial_config.yaml	全局配置（模型、阈值、seed 等）
│ └─ arms/	9个 Arm 的 prompt 配置
│ └─ stage1_A1a.yaml	
│ └─ stage1_A1b.yaml	
│ └─ stage1_A1c.yaml	
│ └─ stage2_A2a.yaml	
│ └─ stage2_A2b.yaml	
│ └─ stage2_A2c.yaml	
│ └─ stage3_A3a.yaml	
│ └─ stage3_A3b.yaml	
│ └─ stage3_A3c.yaml	
├─ core/	
│ └─ patient_agent.py	MediQ Fact-Select 封装
│ └─ doctor_agent.py	MediQ Expert + Arm 注入层
│ └─ judge.py	R1 / R2 / Outcome 评估
│ └─ orchestrator.py	主控逻辑
│ └─ randomizer.py	分层随机 + Re-randomization
├─ data/	
│ └─ loader.py	加载 iMEDQA, 提取/推断字段
│ └─ red_flag_cache.json	离线生成的精细 red flags (可选)
├─ models/	
│ └─ model_client.py	统一模型调用接口
├─ logging/	
│ └─ trajectory_logger.py	JSONL 记录
├─ outputs/	
│ └─ encounters/	每个 encounter 的 JSONL 文件
├─ scripts/	
│ └─ generate_red_flags.py	离线为 iMEDQA 生成 case 级 red flags
│ └─ summarize_encounters.py	读 JSONL, 输出统计摘要
├─ tests/	
│ └─ test_patient.py	
│ └─ test_judge.py	
│ └─ test_encounter.py	
└─ run_encounter.py	入口脚本